МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий фізико-технічний інститут

Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Кіберзахист об’єктів критичної інфраструктури»

Лабораторна робота № 4. Використання симулятора Cooja для моделювання атак на об’єкти критичної інфраструктури

Варіант 3

Виконав: Вовкотруб Олександр Віталійович
Група: ФБ-61мн

Перевірив: Войцеховський А. В.

Київ 2026

Зміст

1 Мета роботи **2**

2 Постановка задачі **2**

3 Теоретичні відомості **2**

4 Хід виконання **2**

4.1 Моделювання шкідливого вузла (п. 9-11) 2

4.2 Сценарії моделювання 3

4.3 Обробка даних 4

5 Результати **5**

5.1 Вплив на легітимні вузли 5

6 Аналіз результатів **5**

6.1 Додатковий сценарій: радіо завжди увімкнене 6

7 Відповіді на контрольні питання **7**

8 Висновки **7**

9 Перелік джерел **8**

Додаток А. Порівняння енергоспоживання (`solution.py`) **9**

1 Мета роботи

Ознайомитися з можливостями симулятора Cooja для моделювання атаки на емульовану мережу безпровідних пристроїв об’єкта критичної інфраструктури, зокрема DIS-атаки на протокол маршрутизації RPL, та оцінити її вплив на споживання енергії вузлами.

2 Постановка задачі

Відповідно до варіанта 3 необхідно змоделювати мережу з 7 вузлів-відправників і 1 приймача (як у роботі № 3), додати **шкідливий вузол**, що реалізує DIS-атаку, і порівняти споживання енергії вузлів (Power, Listen Power, CPU Power) до та після атаки.

3 Теоретичні відомості

У мережах об’єктів критичної інфраструктури використовується протокол маршрутизації **RPL** для малопотужних мереж з втратами. RPL будує топологію у вигляді орієнтованого ациклічного графа (DODAG) з коренем у приймачі. Кожному вузлу присвоюється ранг (Rank), що зростає з віддаленням від кореня; вузли обирають маршрут з найменшим рангом.

Серед керуючих повідомлень RPL є **DIS** (DODAG Information Solicitation, ICMPv6) - запит інформації від сусіднього DODAG, аналог запиту маршрутизатора. У нормальному режимі вузол надсилає DIS лише доки не долучиться до DODAG, з інтервалом `RPL_DIS_INTERVAL` (60 с) і початковою затримкою `RPL_DIS_START_DELAY` (5 с).

DIS-атака - це пряма атака на мережеві ресурси. Вузол під контролем зловмисника безперервно (широкомовно) розсилає службові повідомлення DIS. Кожне DIS змушує сусідів скинути таймери Trickle і відповісти повідомленням DIO, що збільшує обсяг керуючого трафіку, навантаження на процесор і час роботи радіо, а отже - споживання енергії. Тому DIS-атаку відносять до класу DoS-атак (відмова в обслуговуванні через виснаження ресурсів та енергії).

4 Хід виконання

Середовище те саме, що в роботі № 3 (Contiki 2.7, msp430-gcc, Cooja).

4.1 Моделювання шкідливого вузла (п. 9-11)

Для імітації діяльності шкідливого вузла, що постійно надсилає DIS, змінено два файли ядра ContikiRPL (попередньо збережено копію каталогу `core/net/rpl`).

Файл `rpl-private.h` (рис. 6.1 методички) визначає приватні декларації ContikiRPL. Інтервал DIS-повідомлень і таймер затримки запуску встановлено в нуль:

```
 /* DIS related */
 #define RPL_DIS_SEND                    1
 #ifdef  RPL_DIS_INTERVAL_CONF
 #define RPL_DIS_INTERVAL                RPL_DIS_INTERVAL_CONF
 #else
-#define RPL_DIS_INTERVAL                60
+#define RPL_DIS_INTERVAL                0
 #endif
-#define RPL_DIS_START_DELAY             5
+#define RPL_DIS_START_DELAY             0
```

Файл `rpl-timers.c` (рис. 6.2 методички) керує таймерами. У функції `handle_periodic_timer` прибрано умову, що зупиняла надсилання DIS після долучення до DODAG, тому вузол розсилає DIS безперервно (щосекунди, бо періодичний таймер спрацьовує кожну секунду, а `next_dis >= 0` завжди істинне):

```
   /* handle DIS */
 #ifdef RPL_DIS_SEND
   next_dis++;
-  if(rpl_get_any_dag() == NULL && next_dis >= RPL_DIS_INTERVAL) {
+  if(next_dis >= RPL_DIS_INTERVAL) {
     next_dis = 0;
     dis_output(NULL);
   }
 #endif
```

Реалізовано **модель одного зловмисника** (як і в методичці, де шкідливим є окремий вузол `udp-sender2`): з модифікованим RPL скомпільовано лише прошивку `udp-sender2.sky`, а решта вузлів (приймач і 7 легітимних відправників) використовують нормальний RPL (файли ядра відновлено з копії перед їх компіляцією).

4.2 Сценарії моделювання

Виконано два запуски з однаковою топологією і тривалістю (660 с модельного часу), дані збиралися приймачем так само, як у роботі № 3:

- **до атаки** - 7 легітимних відправників (вузли 2…8) + 1 приймач (вузол 1);
- **після атаки** - ті самі вузли + шкідливий вузол 9 (прошивка `udp-sender2.sky`), розміщений у центрі мережі поряд з приймачем і вузлами 2, 3 (рис. 6.1). Вузол 9 постійно розсилає DIS.

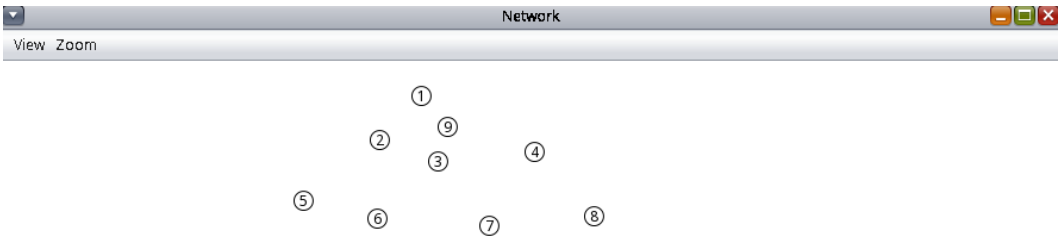


Рисунок 6.1. Топологія після додавання шкідливого вузла 9

Під час атаки симуляція показує інтенсивний радіообмін; вікно Timeline відображає роботу 9 вузлів, приймач продовжує отримувати телеметрію (рис. 6.2).

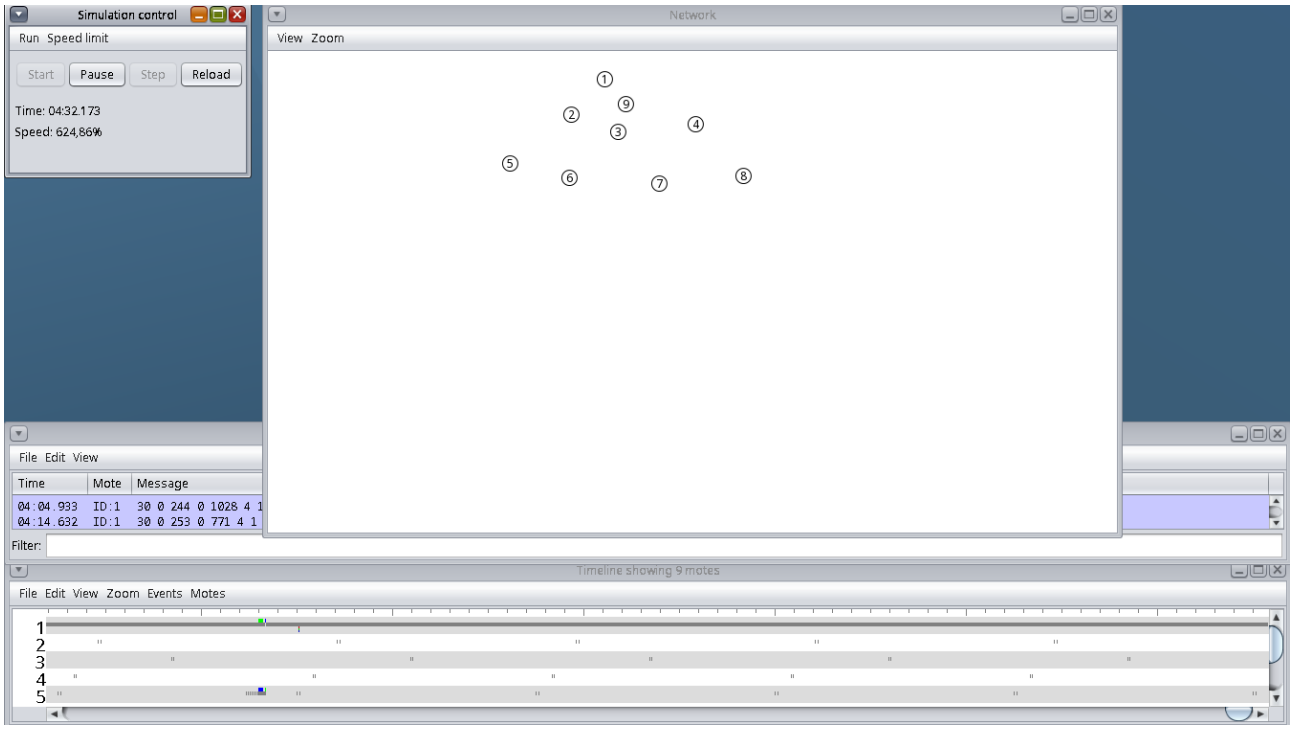


Рисунок 6.2. Запущена симуляція з DIS-атакою (9 вузлів, Timeline)

4.3 Обробка даних

Журнали обох запусків (`data/lab3-baseline.log`, `data/lab4-attack.log`) оброблено скриптом `solution.py`, який за лічильниками energest обчислює середню потужність кожного легітимного вузла (формули Collect View, як у роботі № 3) і порівнює стани до та після атаки.

5 Результати

5.1 Вплив на легітимні вузли

Порівняння споживаної потужності вузлів 2…8 до та після атаки наведено в табл. 6.1 і на рис. 6.3.

Таблиця 6.1. Споживана потужність легітимних вузлів до/після атаки, мВт

Вузол	Power до	Power після	Δ, %	Listen до	Listen після	CPU до	CPU після
2.2	0,984	1,026	+4,2	0,402	0,425	0,380	0,391
3.3	0,986	1,011	+2,5	0,402	0,411	0,387	0,400
4.4	0,975	0,984	+0,9	0,401	0,402	0,376	0,387
5.5	0,976	0,995	+2,0	0,396	0,416	0,365	0,375
6.6	0,991	1,018	+2,7	0,408	0,409	0,388	0,399
7.7	1,008	1,015	+0,7	0,410	0,407	0,388	0,405
8.8	0,965	0,980	+1,5	0,398	0,401	0,362	0,373
Середнє	**0,984**	**1,004**	**+2,1**	**0,402**	**0,410**	**0,378**	**0,390**

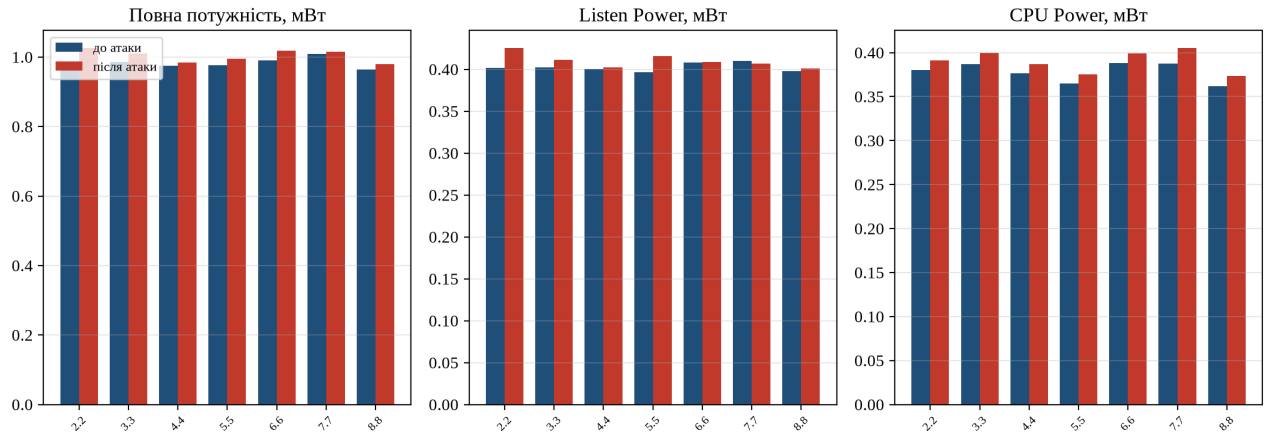


Рисунок 6.3. Споживана потужність вузлів до (синій) та після (червоний) DIS-атаки

6 Аналіз результатів

Вплив на мережу. Атака збільшує споживання енергії всіх семи легітимних вузлів без винятку (рис. 6.3): у середньому повна потужність зростає на 2,1 %, CPU на 3,2 %, Listen на 1,9 %. Найбільше зростання у вузла 2 (+4,2 %), найближчого до зловмисника; решта вузлів показують невелике зростання в межах 0,7…2,7 % без чіткої залежності від відстані. Зростання саме Listen і CPU (а не Transmit) підтверджує механізм атаки: вузли змушені приймати й обробляти зайві DIS-повідомлення та відповідати на них повідомленнями DIO, тримаючи радіо увімкненим довше.

Передача шкідливого вузла. Потужність передачі вузла 9 (0,056 мВт) не виділяється на тлі легітимних вузлів (0,043…0,062 мВт). Це наслідок того, що ContikiMAC дуплексує й обмежує навіть флуд: реальні радіопередачі DIS розтягуються в часі, а не йдуть суцільним потоком. Тому за самою лише потужністю передачі відрізнити зловмисника не вдається.

Чому ефект помірний. Один зловмисник, що розсилає DIS приблизно щосекунди кільком сусідам, додає відносно небагато трафіку, тому зростання енергоспоживання становить одиниці відсотків. Роль дуплексування радіо ContikiMAC перевірено окремим сценарієм (п. нижче): вимкнення duty cycling не підсилює атаку, а лише піднімає базову потужність. Для автономних батарейних сенсорів навіть постійне зростання на 2…4 % скорочує час роботи, а тривала атака лишається загрозою.

Наслідки для OKI. Навіть помірне, але постійне зростання споживання скорочує час автономної роботи батарейних сенсорів (датчиків тиску, рівня, температури). Оцінка для типової батареї з двох елементів AA (2500 мА·год, послідовно 3 В, тобто 7,5 Вт·год) дає для середнього вузла 318 діб роботи до атаки проти 311 діб під постійною атакою: лише ця помірна атака скорочує термін служби приблизно на 6 діб (2 %), а сильніший флуд або кілька зловмисників зменшують цей час пропорційно доданому трафіку. Виснаження й вихід таких вузлів з ладу означає втрату спостережуваності технологічного процесу, тому для критичної інфраструктури DIS-флуд лишається загрозою, а виявити його за самою потужністю важко.

6.1 Додатковий сценарій: радіо завжди увімкнене

Щоб перевірити роль ContikiMAC, обидва запуски повторено з вимкненим дуплексуванням радіо (команда `mac 0`, яку надсилає й сам Collect View при Start Collect; радіо весь час увімкнене). Результати (табл. 6.2) неочікувані, але показові.

Таблиця 6.2. Середня потужність легітимних вузлів, радіо завжди увімкнене, мВт

Показник	До атаки	Після атаки	Δ
Повна потужність	56,10	56,07	−0,0 %
Listen Power	55,50	55,46	−0,1 %
CPU Power	0,390	0,400	+2,8 %

Без дуплексування повна потужність зростає з ~0,9 до ~56 мВт (у 60 разів), бо радіо постійно слухає, і цей постійний Listen повністю домінує над усім іншим. Через це DIS-атака майже **не змінює** повну потужність (−0,0 %): вузли й так не сплять, тож флуд не може змусити їх «не спати більше». Залишається тільки зростання CPU (+2,8 %) від обробки зайвих DIS, таке саме, як у сценарії з ContikiMAC.

Висновок уточнює очікування методички: вимкнення duty cycling не підсилює DIS-атаку, а лише піднімає базову «вартість» мережі; відносний вплив атаки на енергію лишається помірним в обох режимах. Єдина стабільна енергетична ознака DIS-флуду, це невелике (близько 3 %) зростання CPU Power, а не різкий стрибок повної потужності.

7 Відповіді на контрольні питання

1. Призначення файлів `rpl-private.h` та `rpl-timers.c`. `rpl-private.h` містить приватні декларації ContikiRPL: значення за замовчуванням для керуючих повідомлень ICMPv6 (зокрема параметри DIS: `RPL_DIS_INTERVAL`, `RPL_DIS_START_DELAY`), пов'язані таймери, режими роботи, константи таблиць маршрутизації DAG. `rpl-timers.c` реалізує керування таймерами ContikiRPL: періодичний таймер (`handle_periodic_timer`), таймери Trickle для DIO, надсилання DIS та DAO. Саме тут приймається рішення про надсилання DIS.

2. Сутність DIS-атаки для безпровідної мережі. Зловмисний вузол безперервно розсилає широкомовні DIS-повідомлення (запити інформації про DODAG). Кожне DIS змушує вузли-сусіди скидати таймери Trickle і відповідати DIO, що різко збільшує керуючий трафік, навантаження на процесор і час роботи радіо. Це виснажує енергію (особливо критично для батарейних пристроїв), створює затримки й може призвести до відмови в обслуговуванні вузлів мережі.

3. Які зміни треба внести до `rpl-private.h` та `rpl-timers.c` для моделювання DIS-атаки? У `rpl-private.h` встановити `RPL_DIS_INTERVAL` і `RPL_DIS_START_DELAY` у 0 (DIS надсилатиметься без початкової затримки, а далі щосекунди, на кожному спрацюванні періодичного таймера). У `rpl-timers.c` у функції `handle_periodic_timer` прибрати умову `rpl_get_any_dag() == NULL`, щоб вузол надсилав DIS постійно, а не лише доки не долучиться до DODAG. Скомпілювати з цими змінами прошивку шкідливого вузла.

4. Як можна розпізнати здійснення DIS-атаки в мережі OKI? За аномально високою частотою DIS-повідомлень від одного вузла (аналіз трафіку, IDS для RPL); за різким зростанням керуючого трафіку (DIS/DIO) у сегменті; за аномальним зростанням енергоспоживання та часу роботи радіо (Listen/CPU Power) сусідніх вузлів; за частими скиданнями таймерів Trickle і повторними приєднаннями до DODAG; за збільшенням затримок доставки даних. Засоби захисту: обмеження частоти DIS, автентифікація керуючих повідомлень (Secure RPL), виявлення аномалій трафіку та енергоспоживання.

8 Висновки

У симуляторі Cooja реалізовано DIS-атаку на мережу RPL: змінами у файлах ядра ContikiRPL (`rpl-private.h`, `rpl-timers.c`) скомпільовано шкідливий вузол, що безперервно розсилає DIS, тоді як решта мережі працює на нормальному RPL. Атака підвищила споживання енергії всіх легітимних вузлів (у середньому на 2,1 %, CPU на 3,2 %), найбільше у найближчого до зловмисника (+4,2 %); зростання Listen і CPU підтверджує механізм виснаження ресурсів через обробку зайвих DIS. У термінах автономності це скорочує час роботи вузла від двох батарей AA з 318 до 311 діб (−6 діб, або −2 % від терміну служби). Головний висновок стосується захисту: механізм ContikiMAC дає мережі відчутну стійкість до DIS-флуду за енергоспоживанням (ефект обмежений одиницями відсотків, а передача зловмисника не виділяється на тлі інших), тому за самою потужністю виявити атаку важко; потрібні інші ознаки, перелічені у відповіді на питання 4. Додатковий запуск з вимкненим дуплексуванням

радіо показав, що це не підсилює атаку, а лише піднімає базову потужність у ~60 разів; відносний вплив DIS-флуду лишається помірним в обох режимах, а його стабільною ознакою є невелике (близько 3 %) зростання CPU Power.

9 Перелік джерел

1. Winter T. et al. RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks : RFC 6550. IETF, 2012.
2. Mayzaud A., Badonnel R., Chrisment I. A Taxonomy of Attacks in RPL-based Internet of Things. Int. J. of Network Security, 2016.
3. Osterlind F., Dunkels A., Eriksson J. et al. Cross-level sensor network simulation with COOJA. LCN 2006.
4. Dunkels A. et al. Software-based on-line energy estimation for sensor nodes (Energest). EmNets 2007.
5. Стьопочкіна І. В., Ільїн К. І., Овчарук М. В. Кіберзахист об’єктів критичної інфраструктури. Лабораторний практикум : навч. посіб. Київ : КПІ ім. Ігоря Сікорського, 2023. 59 с.

Додаток А. Порівняння енергоспоживання (`solution.py`)

```
"""ЛР4: порівняння споживання енергії до та після DISатаки- на RPL.Обробляє

 два журнали Cooja: baseline (7 sender + 1 sink) і attack той( самийсценарій
 + шкідливий вузол, що безперервно розсилає DIS). Рахує потужністькожного
 легітимного вузла за energestлічильниками- формули( Collect View) ібудує
 порівняння допісля/ для Power, Listen Power, CPU Power.

    ../../.venv/bin/python solution.py
"""

import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

HERE = Path(__file__).resolve().parent
ASSETS = HERE / "assets"
DATA = HERE / "data"

VOLTAGE = 3.0
POWER_CPU = 1.800 * VOLTAGE
POWER_LPM = 0.0545 * VOLTAGE
POWER_TRANSMIT = 17.7 * VOLTAGE
POWER_LISTEN = 20.0 * VOLTAGE
NODE_ID, TIME_CPU, TIME_LPM, TIME_TX, TIME_LISTEN = 4, 11, 12, 13, 14


def node_name(raw: int) -> str:
    return f"{raw & 0xff}.{(raw >> 8) & 0xff}"


def parse_log(path: Path) -> dict[str, list[list[int]]]:
    readings: dict[str, list[list[int]]] = {}
    for line in path.read_text().splitlines():
        p = line.split()
        if len(p) < 3 or not p[1].startswith("ID:") or p[2] != "30":
            continue
        vals = [int(x) for x in p[2:]]
        if len(vals) == 30:
            readings.setdefault(node_name(vals[NODE_ID]), []).append(vals)
    return readings
```

```
def power(vals: list[int]) -> dict[str, float]:
    cpu, lpm, tx, lst = vals[TIME_CPU], vals[TIME_LPM], vals[TIME_TX], vals[TIME_LISTEN]
    d = cpu + lpm
    pc, pl, pt, pls = (cpu * POWER_CPU / d, lpm * POWER_LPM / d,
                       tx * POWER_TRANSMIT / d, lst * POWER_LISTEN / d)
    return {"cpu": pc, "lpm": pl, "tx": pt, "listen": pls, "total": pc + pl + pt + pls}


def avg_power(readings: dict[str, list[list[int]]], nodes: list[str]) -> dict[str, dict]:
    out = {}
    for n in nodes:
        # усереднюємо потужність по всіх звітах вузла (energest кумулятивний у вікні звіту)
        ps = [power(v) for v in readings[n]]
        out[n] = {k: float(np.mean([p[k] for p in ps])) for k in ps[0]}
    return out


def main() -> None:
    base = parse_log(DATA / "lab3-baseline.log")
    atk = parse_log(DATA / "lab4-attack.log")
    # легітимні вузливідправники-: ті, що є в обох запусках (2.2 ... 8.8)
    legit = sorted(set(base) & set(atk),
                   key=lambda s: tuple(map(int, s.split("."))))
    attacker = sorted(set(atk) - set(base))  # 9.9

    pb = avg_power(base, legit)
    pa = avg_power(atk, legit)

    rows = []
    for n in legit:
        rows.append({
            "node": n,
            "total_before": pb[n]["total"], "total_after": pa[n]["total"],
            "listen_before": pb[n]["listen"], "listen_after": pa[n]["listen"],
            "cpu_before": pb[n]["cpu"], "cpu_after": pa[n]["cpu"],
            "total_delta_pct": 100 * (pa[n]["total"] - pb[n]["total"]) / pb[n]["total"],
        })

    summary = {
        "legit_nodes": legit,
        "attacker": attacker,
        "mean_total_before": float(np.mean([r["total_before"] for r in rows])),
        "mean_total_after": float(np.mean([r["total_after"] for r in rows])),
        "mean_listen_before": float(np.mean([r["listen_before"] for r in rows])),
        "mean_listen_after": float(np.mean([r["listen_after"] for r in rows])),
        "mean_cpu_before": float(np.mean([r["cpu_before"] for r in rows])),
        "mean_cpu_after": float(np.mean([r["cpu_after"] for r in rows])),
        "rows": rows,
    }
```

```
# ---- час автономної роботи від батарей (2 x AA, ~2500 мАгод*, 3 В) ----
# E = 2 * 2.5 Агод* * 1.5 В = 7.5 Втгод*; T = E / P
BATTERY_WH = 2 * 2.5 * 1.5
t_b = BATTERY_WH / summary["mean_total_before"] * 1e3 / 24   # дні
t_a = BATTERY_WH / summary["mean_total_after"] * 1e3 / 24
summary["battery"] = {
    "capacity_wh": BATTERY_WH,
    "lifetime_days_before": t_b,
    "lifetime_days_after": t_a,
    "lifetime_loss_days": t_b - t_a,
    "lifetime_loss_pct": 100 * (t_b - t_a) / t_b,
}

(DATA / "compare.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2))

# ---- графік: допісля/ для Total, Listen, CPU ----
plt.rcParams.update({"font.family": "Liberation Serif", "font.size": 11})
fig, axes = plt.subplots(1, 3, figsize=(12, 4.3), sharey=False)
names = [r["node"] for r in rows]
xs = np.arange(len(rows))
w = 0.38
for ax, (key, title) in zip(axes, [("total", "Повна потужність"),
                                   ("listen", "Listen Power"),
                                   ("cpu", "CPU Power")]):
    b = [pb[n][key] for n in names]
    a = [pa[n][key] for n in names]
    ax.bar(xs - w / 2, b, w, label="до атаки", color="#1f4e79")
    ax.bar(xs + w / 2, a, w, label="після атаки", color="#c0392b")
    ax.set_title(title + ", мВт")
    ax.set_xticks(xs, names, fontsize=8, rotation=45)
    ax.grid(alpha=0.3, axis="y")
axes[0].legend(loc="upper left", fontsize=9)
fig.tight_layout()
ASSETS.mkdir(exist_ok=True)
fig.savefig(ASSETS / "03-power-compare.png", dpi=200)

print(f"legit nodes: {legit}")
print(f"attacker: {attacker}")
print(f"{'node':6} {'tot_bef':>8} {'tot_aft':>8} Δ{'%':>7} {'lst_bef':>8} {'lst_aft':>8}
↪ {'cpu_bef':>8} {'cpu_aft':>8}")
for r in rows:
    print(f"{r['node']:6} {r['total_before']:8.3f} {r['total_after']:8.3f} "
          f"{r['total_delta_pct']:7.1f} {r['listen_before']:8.3f} {r['listen_after']:8.3f}
↪  "
          f"{r['cpu_before']:8.3f} {r['cpu_after']:8.3f}")
print(f"MEAN   {summary['mean_total_before']:8.3f} {summary['mean_total_after']:8.3f}"
      f"   listen {summary['mean_listen_before']:.3f}->{summary['mean_listen_after']:.3f}"
      f"   cpu {summary['mean_cpu_before']:.3f}->{summary['mean_cpu_after']:.3f}")
```

```
    b = summary["battery"]
    print(f"battery: {b['lifetime_days_before']:.0f} -> {b['lifetime_days_after']:.0f} днів "
          f"(-{b['lifetime_loss_days']:.0f} днів, -{b['lifetime_loss_pct']:.1f} %)")


if __name__ == "__main__":
    main()
```